

SOCShield

**Intelligent Web Attack Detection,
Threat Intelligence, and Automated Incident
Response Platform.**

Technical Project Documentation

**A Security Operations Center (SOC) platform for
real-time web attack detection, threat intelligence
enrichment, and automated incident response.**

Version: 1.0

**Author:
Om Dhumal**

**Technologies:
Wazuh SIEM | ModSecurity WAF | NGINX | DVWA
Burp Suite | VirusTotal | MITRE ATT&CK
Postfix | Linux**

Date: May 2026

- **Table of Contents:**

- 1. Introduction**
- 2. Problem Statement**
- 3. Objectives**
- 4. Tools and Technologies**
- 5. Solution Architecture**
- 6. Implementation**
- 7. Attack Simulation**
- 8. MITRE ATT&CK Mapping**
- 9. Active Response**
- 10. Threat Intelligence Integration**
- 11. Email Alerting**
- 12. Dashboard & Monitoring**
- 13. Results**
- 14. Challenges Encountered**
- 15. Future Enhancements**
- 16. Conclusion**

Executive Summary

Organizations increasingly face sophisticated cyber threats targeting web applications, including Cross-Site Scripting (XSS),

SQL Injection (SQLi), and Remote Code Execution (RCE) attacks. Traditional monitoring approaches often lack centralized visibility, threat intelligence enrichment and automated response capabilities, making timely detection and mitigation difficult.

SOCShield is an intelligent Security Operations Center (SOC) platform designed to provide real-time web attack detection, threat intelligence integration, and automated incident response. The platform combines multiple open-source security technologies, including Wazuh SIEM, ModSecurity Web Application Firewall (WAF), NGINX, DVWA, VirusTotal, MITRE ATT&CK Mapping, Postfix Email Alerting and Linux-based monitoring components.

The solution continuously monitors web application activity, collects and analyzes security logs, identifies malicious behavior, correlates security events, and generates actionable alerts. Attack simulations were performed using Kali Linux and Burp Suite to validate the detection of Cross-Site Scripting (XSS), SQL Injection (SQLi), and Remote Code Execution (RCE) attacks. Detected events are mapped to relevant MITRE ATT&CK techniques to improve threat visibility and investigation capabilities.

To enhance incident response, SOCShield integrates automated firewall-based active response mechanisms capable of blocking

malicious source IP addresses. The platform also incorporates VirusTotal threat intelligence enrichment and real-time email alerting to improve analyst awareness and response efficiency.

A centralized dashboard provides security monitoring, threat hunting, attack visualization, MITRE ATT&CK mapping, attacker IP analysis, alert severity monitoring, and incident investigation capabilities. The implementation demonstrates how open-source security technologies can be integrated to build a practical and cost-effective SOC environment for web application security monitoring and incident response.

The results confirm the successful detection, analysis, alerting, and response to multiple web-based attack scenarios, making SOCSHield a comprehensive platform for modern security operations and threat management.

1. Introduction

In today's digital landscape, organizations rely heavily on web applications and online services to support business operations, customer interactions, and data management. While these technologies provide significant advantages, they also introduce a growing attack surface that can be exploited by cybercriminals. Web-based attacks such as Cross-Site Scripting (XSS), SQL Injection (SQLi), and Remote Code Execution (RCE) continue to be among the most common and impactful threats targeting modern applications.

As cyber threats become increasingly sophisticated, organizations require more than traditional security controls. Modern security operations demand continuous monitoring, centralized log analysis, threat intelligence integration, attack correlation, and automated incident response capabilities. Security Operations Centers (SOCs) play a critical role in achieving these objectives by providing visibility into security events and enabling security teams to detect, investigate, and respond to threats in real time.

Security Information and Event Management (SIEM) platforms have become essential components of SOC environments. SIEM solutions collect logs from multiple sources, correlate events, generate alerts, and provide centralized monitoring capabilities. When combined with Web Application Firewalls (WAFs), threat intelligence platforms, and automated response mechanisms, SIEM systems significantly improve an organization's ability to identify and mitigate cyber threats.

SOCShield is designed as an intelligent web attack detection, threat intelligence, and automated incident response platform built using open-source cybersecurity technologies. The platform integrates Wazuh SIEM, ModSecurity Web Application Firewall (WAF), NGINX Web Server, DVWA (Damn Vulnerable Web Application), VirusTotal Threat Intelligence, MITRE ATT&CK Mapping, Postfix Email Alerting, and Linux-based monitoring components into a unified security monitoring environment.

The platform continuously monitors web application activity, collects security logs, detects malicious behavior, enriches alerts with threat intelligence data, maps attacks to the MITRE ATT&CK framework, and performs automated response actions when security incidents are identified. To validate the effectiveness of the platform, multiple attack scenarios including Cross-Site Scripting (XSS), SQL Injection (SQLi), and Remote Code Execution (RCE) were simulated using Kali Linux and Burp Suite.

The primary goal of SOCSshield is to demonstrate how open-source security technologies can be integrated to create a practical, scalable, and cost-effective Security Operations Center capable of detecting, analyzing, and responding to modern web-based cyber threats in real time.

2. Problem Statement

Modern organizations increasingly depend on web applications to support business operations, customer services, and data management. While web technologies provide flexibility and accessibility, they also introduce significant security risks. Cyber attackers frequently target web applications using techniques such as Cross-Site Scripting (XSS), SQL Injection (SQLi), and Remote Code Execution (RCE) to gain unauthorized access, manipulate data, or disrupt services.

Many organizations rely on traditional security monitoring approaches that generate large volumes of logs but provide limited visibility into actual security threats. Security teams often face challenges in identifying malicious activities across multiple systems, correlating security events, and responding to incidents in a timely manner. In addition, the absence of centralized monitoring, threat intelligence integration, and automated response mechanisms can significantly increase the time required to detect and mitigate attacks.

Web Application Firewalls (WAFs) and logging solutions can generate valuable security data; however, without proper analysis, correlation, and visualization, organizations may struggle to transform raw logs into actionable security insights. Furthermore, many environments lack automated incident response capabilities, resulting in delayed containment of malicious activities and increased security risks.

To address these challenges, there is a need for a unified Security Operations Center (SOC) platform capable of continuously monitoring web application activity, detecting attacks in real time, correlating security events, enriching alerts with threat intelligence, and automatically responding to malicious activities. Such a platform should provide centralized visibility, improve threat detection accuracy, reduce incident response time, and assist security analysts in investigating and managing security incidents effectively.

SOCShield was developed to address these challenges by integrating Security Information and Event Management (SIEM), Web Application Firewall (WAF) monitoring, threat intelligence enrichment, MITRE ATT&CK mapping, automated response mechanisms, and real-time alerting into a single security monitoring platform. The solution aims to enhance an organization's ability to detect, analyze, and respond to modern web-based cyber threats efficiently and effectively.

3. Objectives

The primary objective of SOCSshield is to design and implement an intelligent Security Operations Center (SOC) platform capable of detecting, analyzing, and responding to web-based cyber threats in real time. The platform aims to enhance security visibility, improve incident response capabilities, and demonstrate the effective integration of open-source cybersecurity technologies.

The specific objectives of the project are as follows:

1. To develop a centralized security monitoring platform using Wazuh SIEM for collecting, analyzing, and correlating security events.
2. To deploy and configure ModSecurity Web Application Firewall (WAF) with OWASP Core Rule Set (CRS) for detecting and mitigating web application attacks.
3. To simulate real-world web attacks, including Cross-Site Scripting (XSS), SQL Injection (SQLi), and Remote Code Execution (RCE), using Kali Linux and Burp Suite.
4. To create custom detection rules for identifying and classifying web-based attacks within the Wazuh environment.
5. To integrate MITRE ATT&CK Mapping for improved threat visibility and attack classification.

6. To implement Active Response mechanisms capable of automatically blocking malicious source IP addresses through firewall-drop actions.
7. To integrate VirusTotal Threat Intelligence services for alert enrichment and reputation-based threat analysis.
8. To implement real-time email alerting for notifying security analysts about critical security incidents.
9. To develop interactive SOC dashboards for threat hunting, attack visualization, incident monitoring, and security analysis.
10. To demonstrate a practical and cost-effective approach for building a modern Security Operations Center using open-source technologies.
11. To improve the overall capability of detecting, investigating, and responding to web-based cyber threats through centralized monitoring and automated response mechanisms.

4. Tools and Technologies

SOCShield was developed using a combination of open-source cybersecurity, monitoring, web security, and threat intelligence technologies. Each component was selected to provide a specific security function within the overall architecture, enabling attack detection, threat analysis, automated response, and centralized monitoring.

Tool/Technology	Category	Purpose in Project
Kali Linux based attack	Attack Simulation	Used to simulate web-
Site Scripting (XSS),	Platform	such as Cross-
Injection (SQLi), and Remote		SQL
Execution (RCE).		Code
Burp Suite modifying,	Web Application Testing	Used for intercepting,
HTTP requests during	Tool	and replaying
simulation.		attack
DVWA (Damn application for	Vulnerable Web	Served as the target
Vulnerable Web attack	Application	security testing and
Application)		simulation.
Ubuntu Server application, WAF	Operating System	Hosted the web
Components, monitoring		SIEM

Infrastructure.

NGINX
and processed

Web Server

Hosted DVWA

Incoming

HTTP requests before

Inspection by ModSecurity.

ModSecurity
and detected

Web Application

Inspected web traffic

like XSS, SQLi etc

Firewall

malicious request

OWASP Core Rule
attack detection

Security Rules

Provided predefined

Set (CRS
web application

Firewall (WAF)

rules for common

threats.

Wazuh Agent
security events

Endpoint Monitoring

Collected logs and

monitored server

Component

from the

environment.

Wazuh Manager
event

Security Information

Performed log collection,

matching, and

and Event Management

correlation, rule

(SIEM)

alert

generation.

Wazuh Indexer
security events, and

Data Storage and

Stored alerts,

(OpenSearch)
analysis and

Search Engine

monitoring data for

visualization.

Wazuh Dashboard
threat

Security Monitoring

Provided dashboards,

Interface

hunting

capabilities, attack

visualization incident monitoring.

MITRE ATT&CK
detected attacks to

Threat Classification

Used to map

Framework
tactics

Framework

known adversarial

and

techniques.

VirusTotal
alerts with

Threat Intelligence

Enriched security

Framework

threat

intelligence and reputation

Information.

Active Response
malicious

Automated Response

Automatically blocked

(Firewall-Drop)
addresses when

Mechanism

source IP

predefined security conditions

were

met.

Postfix
alert notification

Mail Transfer Agent

Relayed security

(MTA)

to external

email services.

Gmail SMTP
time security

Email Notification

Delivered real-

Service

alerts to

the security analyst.

Technology Integration Overview

The selected technologies were integrated to create a unified Security Operations Center (SOC) environment capable of detecting, analyzing, enriching, and responding to web-based cyber threats. The combination of SIEM, WAF, threat intelligence, automated response, and centralized monitoring components enabled SOCSHield to provide end-to-end visibility into security events while improving incident response efficiency.

5. Solution Architecture

5.1 Architecture Overview

SOCShield is designed as an integrated Security Operations Center (SOC) platform that combines web application security monitoring, threat intelligence enrichment, attack detection, automated response, and centralized visibility into a unified environment.

The architecture consists of multiple interconnected components that work together to identify, analyze, and respond to web-based cyber threats. The platform utilizes open-source technologies including Wazuh SIEM, ModSecurity Web Application Firewall (WAF), NGINX Web Server, VirusTotal Threat Intelligence, MITRE ATT&CK Mapping, Active Response, and Email Alerting.

The architecture enables continuous monitoring of web application traffic, detection of malicious activities, threat intelligence enrichment, attack classification, automated response actions, and centralized security monitoring through an interactive SOC dashboard.

5.2 Architectural Components

5.2.1 Attack Simulation Layer

The attack simulation layer consists of Kali Linux and Burp Suite. Kali Linux is used to generate attack traffic while Burp Suite is used to intercept, manipulate, and replay HTTP requests

Simulated attacks include:

- Cross-Site Scripting (XSS)
- SQL Injection (SQLi)
- Remote Code Execution (RCE)

This layer provides realistic attack scenarios for validating the effectiveness of the Security monitoring platform.

5.2.2 Web Application Layer

The web application layer consists of DVWA (Damn Vulnerable Web Application) Hosted on a NGINX web server.

Responsibilities include:

- Hosting the vulnerable application
- Receiving incoming HTTP requests
- Processing web application traffic

This layer serves as the primary target environment for attack simulations.

5.2.3 Web Application Firewall Layer

ModSecurity integrated with the OWASP Core Set (CRS) acts the Web Application Firewall (WAF)

Responsibilities include:

- Inspection of HTTP requests
- Detection of malicious payloads
- Prevention of common web attacks
- Generation of security events and logs

The WAF provides the first layer of defense against web-based attacks

5.2.4 Security Monitoring layer

The security monitoring layer consists of Wazuh Agent and Wazuh Manager.

Responsibilities include:

- Log Collection
- Event correlation
- Security analysis
- Rule matching
- Alert generation

Security logs from NGINX, ModSecurity and system components are collected and Analyzed to identify malicious activities.

5.2.5 Detection and Correlation Layer

The Detection and Correlation Engine processes security events and applies predefined and custom rules to identify suspicious behavior.

Functions include:

- Attack classification
- Threat detection
- Event correlation
- Alert prioritization

This layer transforms raw security logs into actionable security alerts

5.2.6 Threat Intelligence and Response Layer

The layer enhances detection capabilities through additional security services.

MITRE ATT&CK Mapping

Detected attacks are mapping to MITRE ATT&CK and techniques to provide standardized threat classification.

VirusTotal Integration

Security events are enriched using VirusTotal threat intelligence to improve analyst Visibility and threat assessment.

Active Response

Automated firewall-drop actions are executed to block malicious source IP addresses when predefined conditions are met.

5.2.7 Data Storage and Visualization Layer

Wazuh Indexer (OpenSearch) stores security events, alerts, and monitoring data.

The Wazuh Dashboard provides:

- Threat Hunting
- Security Monitoring
- Severity Heatmaps
- MITRE ATT&CK Visualization
- Top Attacker Analysis
- Incident Investigation

This layer provides centralized visibility into security operations.

5.2.8 Alerting Layer

Postfix SMTP Relay and Gmail SMTP services provide real-time email notifications.

Responsibilities include:

- Security alert delivery
- Incident notifications
- Analyst awareness

Critical security events trigger automatic email notifications to support rapid incident response.

5.3 Architecture Workflow

The Operational workflow of SOCSshield is summarized below:

1. An attacker launches malicious requests using Kali Linux and Burp Suite.
2. Requests are sent to DVWA through the NGINX web server.
3. ModSecurity inspects incoming traffic and identifies malicious payloads.
4. Security events are forwarded to Wazuh Agent and Wazuh Manager.
5. Wazuh performs events are forwarded to Wazuh Agent and Wazuh Manager.
6. Alerts are enriched using VirusTotal and mapped to MITRE ATT&CK techniques.
7. Active Response blocks malicious source IP addresses when required.
8. Security events are stored within the Wazuh Indexer.
9. Analysts monitor activity through the Wazuh Dashboard.
10. Critical incidents generate automated mail notifications via Postfix and Gmail SMTP.

6. Implementation

6.1 Environment Setup

The SOCSshield platform was implemented within a virtualized lab environment using

Oracle VirtualBox. The environment was designed to simulate real-world web application

attacks while providing centralized monitoring, threat detection, and automated response

capabilities.

The implementation consisted of an Ubuntu Server hosting the web application, Web

Application Firewall (WAF), and Security Information and Event Management (SIEM)

components. Kali Linux was used to attack simulation machine to generate malicious

traffic for testing and validation purposes.

Environment Components

- Oracle VirtualBox
- Ubuntu Server
- Kali Linux
- Internal Virtual Network

The virtualized environment provided an isolated and controlled platform for attack simulation, monitoring, and security analysis.

6.2 Web Application Deployment

DVWA (Damn Vulnerable Web Application) was deployed as the target application for attack simulation and security testing.

Deployment Steps

- Installation of required dependencies
- Configuration of web server services
- Deployment of DVWA application files
- Database initialization and configuration
- Validation of web application accessibility

DVWA was selected because it provides intentionally vulnerable functionality that allows security controls to be tested against realistic attack scenarios.

6.3 NGINX Web Server Configuration

NGINX was deployed as the primary web server responsible for handling incoming HTTP requests and serving the DVWA application.

Responsibilities

- Hosting the DVWA application
- Processing HTTP requests
- Logging access and error events

- Forwarding traffic to ModSecurity for inspection

NGINX access and error logs were configured to support security monitoring and event analysis.

6.4 ModSecurity Web Application Firewall Deployment

ModSecurity was implemented as the primary Web Application Firewall (WAF) component.

Configuration Activities

- Installation of ModSecurity
- Integration with NGINX
- Enabling ModSecurity engine
- Configuration of audit logging
- Deployment of OWASP Core Rule Set (CRS)

The WAF was configured to inspect all incoming web requests and identify malicious payloads associated with common web application attacks.

Attack Detection Capabilities

- Cross-Site Scripting (XSS)
 - SQL Injection (SQLi)
 - Remote Code Execution (RCE)
 - Command Injection
 - Directory Traversal
 - Web Enumeration Attempts
-

6.5 Wazuh SIEM Deployment

Wazuh was deployed as the centralized Security Information and Event Management (SIEM) platform.

Components Implemented

- Wazuh Agent
- Wazuh Manager
- Wazuh Indexer
- Wazuh Dashboard

Key Functions

- Log Collection
- Event Correlation
- Security Monitoring
- Alert Generation
- Threat Hunting

The SIEM environment enabled centralized analysis of security events generated by ModSecurity, NGINX and system components.

6.6 Custom Rule Development

Custom detection rules developed to improve visibility into web-based attack activity.

Custom Detection Categories

- XSS Attack Detection
- SQL Injection Detection
- Remote Code Execution Detection
- Web Attack Classification
- Active Response Triggering

These custom rules generated enriched alerts containing attack information, severity levels, source IP addresses, and MITRE ATT&CK mappings.

6.7 MITRE ATT&CK Mapping

MITRE ATT&CK mappings were integrated into custom detection rules to improve attack classification and investigation.

The implementation mapped detected attack activity to relevant MITRE ATT&CK tactics and techniques, providing analysts with standardized threat intelligence and contextual information.

Examples include:

- T1059 – Command and Scripting Interpreter
- T1059.008 – Network Device CLI
- T1083 – File and Directory Discovery

6.8 Active Response Implementation

Wazuh Active Response was configured to automatically respond to high-severity events.

Response Actions

- Detection of malicious source IP addresses
- Automatic firewall-drop execution
- Blocking of attacker systems
- Incident logging and auditing

This capability reduced response times and demonstrated automated incident containment.

6.9 VirusTotal Threat Intelligence Integration

VirusTotal integration was implemented to enrich security alerts with external threat Intelligence data.

Intelligence Capabilities

- Reputation Lookup
- Threat Intelligence Enrichment
- IOC Verification
- Threat Context Enhancement

This integration provided analysts with additional information for evaluating suspicious indicators and potential threats.

6.10 Email Alerting Configuration

Email alerting was implemented using Postfix and Gmail SMTP services.

Alerting Functions

- Real-time alert delivery
- Critical incident notification
- Security event escalation
- Analyst awareness enhancement

Security alerts generated by Wazuh were automatically forwarded via email to improve incident response effectiveness.

6.11 Dashboard Development and Visualization

Custom dashboards were developed within the Wazuh Dashboard to provide centralized visibility into security operations.

Dashboard Features

- Attack Monitoring
- Threat Hunting
- Alert Severity Analysis
- MITRE ATT&CK Visualization
- Active Response Monitoring
- Source IP Analysis

The dashboards enabled security analysts to monitor, investigate, and respond to security incidents through a single interface.

7. Attack Simulation

To validate the effectiveness of SOCSshield, multiple web application attacks were simulated against the DVWA environment. Attack traffic was generated using Kali Linux and Burp Suite, while ModSecurity and Wazuh continuously monitored and analyzed requests.

The objective of these simulations was to verify the platform's ability to detect, classify, alert, and respond to malicious web-based activities in real time.

7.1 Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a web application vulnerability that allows attackers to inject malicious client-side scripts into web pages viewed by other users. Successful exploitation may lead to session hijacking, credentials theft, or unauthorized actions.

Attack Scenario

The XSS module within DVWA was used to simulate reflected XSS attacks. Malicious java script payloads were submitted through vulnerable input fields.

Sample Payload

```
<script>alert('XSS')</script>
```

Detection Process

1. The malicious payload was submitted through DVWA.
2. NGINX received the HTTP request.
3. ModSecurity inspected the request and identified the XSS signature.
4. Security logs were generated and forwarded to Wazuh.
5. Wazuh correlated the event and generated a security alert.
6. The alert was displayed within the Wazuh Dashboar

Results

The attack was successfully detected and classified as a web application attack. SOCSHield generated corresponding alerts and provided visibility through centralized monitoring dashboards.



Figure 7.1: ModSecurity WAF blocked a malicious XSS request and returned an HTTP 403 Forbidden response to the attacker.

```
anonymous@anonymous-VirtualBox:~$ sudo tail -f /var/log/modsec_audit.log
ModSecurity: Access denied with code 403 (phase 2). Matched "Operator 'Ge' with parameter '5' against variable 'TX:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: '23' ) [file "/etc/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "222"] [id "949110"] [rev "" ] [msg "Inbound Anomaly Score Exceeded (Total Score: 23)"] [data "" ] [severity "0"] [ver "OWASP_CRS/4.26.0-dev"] [maturity "0"] [accuracy "0"] [tag "anomaly-evaluation"] [tag "OWASP_CRS"] [hostname "192.168.100.1"] [uri "/DVWA/vulnerabilities/xss_r/"] [unique_id "178077804926.987459"] [ref "" ]
---gkLxqV7z---I--
---gkLxqV7z---J--
---gkLxqV7z---K--
---gkLxqV7z---Z--
█
```

Figure 7.2: ModSecurity audit logs showing detection and blocking of a malicious XSS payload based on OWASP CRS rules.

W.

Threat Hunting

APIdefaulta

116 hits

Jun 7, 2026 @ 02:01:50.408 - Jun 7, 2026 @ 02:05:50.408

Export FormattedReset view520 available fieldsColumnsDensity1 fields sortedExit full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Jun 7, 2026 @ 02:04:12.818	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.818	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.816	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.816	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.816	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:10.818	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.818	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.818	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.817	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.817	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.817	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.816	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.815	anonymous-VirtualBox	ModSecurity XSS Attack Detected	12	100500
Jun 7, 2026 @ 02:04:10.814	anonymous-VirtualBox	XSS (Cross Site Scripting) attempt.	6	31105
Jun 7, 2026 @ 02:04:10.814	anonymous-VirtualBox	ModSecurity rejected a query	7	31333

Figure 7.3: Wazuh SIEM generated security alerts for the detected XSS attack and triggered Active Response actions.

7.2 SQL Injection (SQLi)

SQL Injection is a technique used by attackers to manipulate backend database queries

through unsensitized user input. Successful exploitation may allow unauthorized access to sensitive information or modification of database contents.

Attack Scenario

The SQL Injection module within DVWA was used to simulate malicious database queries.

Sample Payload

' OR '1'='1

Detection Process

1. Malicious SQL input was submitted through DVWA.
2. ModSecurity analyzed the request payload.
3. OWASP CRS rules identified SQL Injection patterns.
4. Security events were forwarded to Wazuh.
5. Wazuh generated an alert and correlated the activity.

Results

SOCShield successfully detected the SQL Injection attempt and generated alerts containing attack information, source IP details, and security classifications.



Figure 7.4: ModSecurity WAF blocked a malicious SQL Injection request and returned an HTTP 403 Forbidden response.

```
---xTVrJ1Km---F---
HTTP/1.1 403
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 06 Jun 2026 21:12:09 GMT
Content-Length: 162
Content-Type: text/html
Connection: keep-alive

---xTVrJ1Km---H---
ModSecurity: Warning. Matched 'Operator 'Rx' with parameter '(?:^([\\d.]|[\\d\\-f:]+)|[\\d\\-f:]+)(:[\\d]+)?$' against variable 'REQUEST_HEADERS:Host' (Value: '192.168.100.1' ) [file "/etc/modsecurity-crs/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf"] [line "756"] [id "920350"] [rev "" ] [msg "Host header is a numeric IP address"] [data "192.168.100.1"] [severity "4"] [ver "OWASP_CRS/4.26.0-dev"] [maturity "0"] [accuracy "0"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-protocol"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "OWASP_CRS/PROTOCOL-ENFORCEMENT"] [tag "capec/1000/210/272"] [hostname "192.168.100.1"] [uri "/DVWA/vulnerabilities/sql/"] [unique_id "178078032952.350572"] [ref "o0,13o0,13v120,13"]
ModSecurity: Warning. detected SQLi using libinjection. [file "/etc/modsecurity-crs/rules/REQUEST-942-APPLICATION-ATTACK-SQLI.conf"] [line "46"] [id "942100"] [rev "" ] [msg "SQL Injection Attack Detected via libinjection"] [data "Matched Data: sos found within A RGS:id: 'XOR%'1='1'] [severity "2"] [ver "OWASP_CRS/4.26.0-dev"] [maturity "0"] [accuracy "0"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-sqli"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "OWASP_CRS/ATTACK-SQLI"] [tag "capec/1000/152/248/66"] [hostname "192.168.100.1"] [uri "/DVWA/vulnerabilities/sql/"] [unique_id "178078032952.350572"] [ref "v35,11"]
ModSecurity: Access denied with code 403 (phase 2). Matched 'Operator 'Ge' with parameter 'S' against variable 'TX:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: '8' ) [file "/etc/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "222"] [id "949110"] [rev "" ] [msg "Inbound Anomaly Score Exceeded (Total Score: 8)"] [data "" ] [severity "0"] [ver "OWASP_CRS/4.26.0-dev"] [maturity "0"] [accuracy "0"] [tag "anomaly-evaluation"] [tag "OWASP_CRS"] [hostname "192.168.100.1"] [uri "/DVWA/vulnerabilities/sql/"] [unique_id "178078032952.350572"] [ref ""]
```

Figure 7.5: ModSecurity audit logs showing SQL Injection detection through OWASP CRS rules and anomaly score evaluation

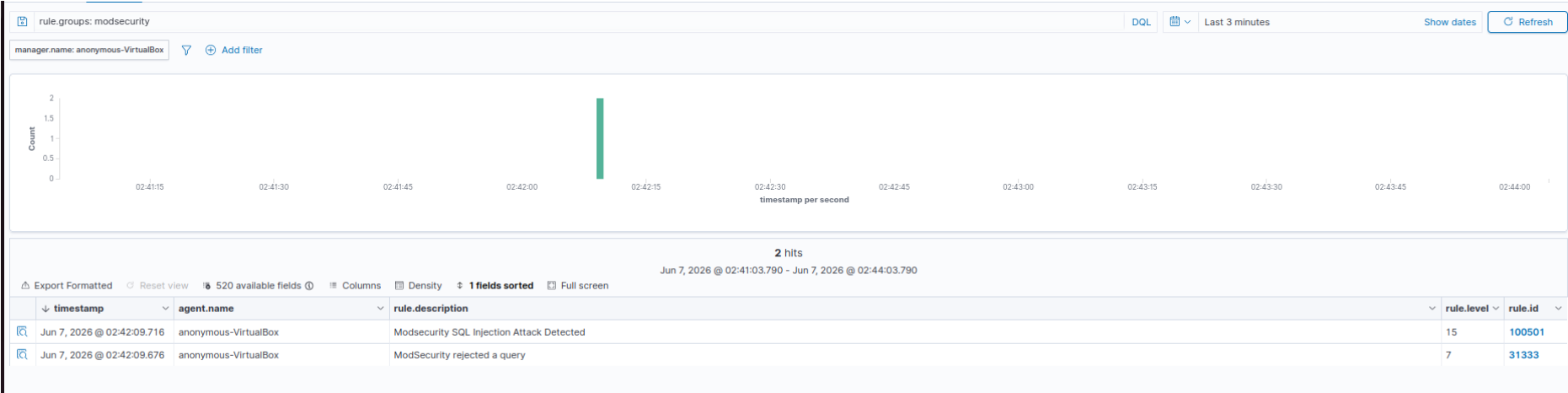


Figure 7.6: Wazuh SIEM generated security alerts for the detected SQL Injection attacks and correlated the event for monitoring and investigation.

Attack Type: SQL Injection

Payload Example: ' OR '1'='1

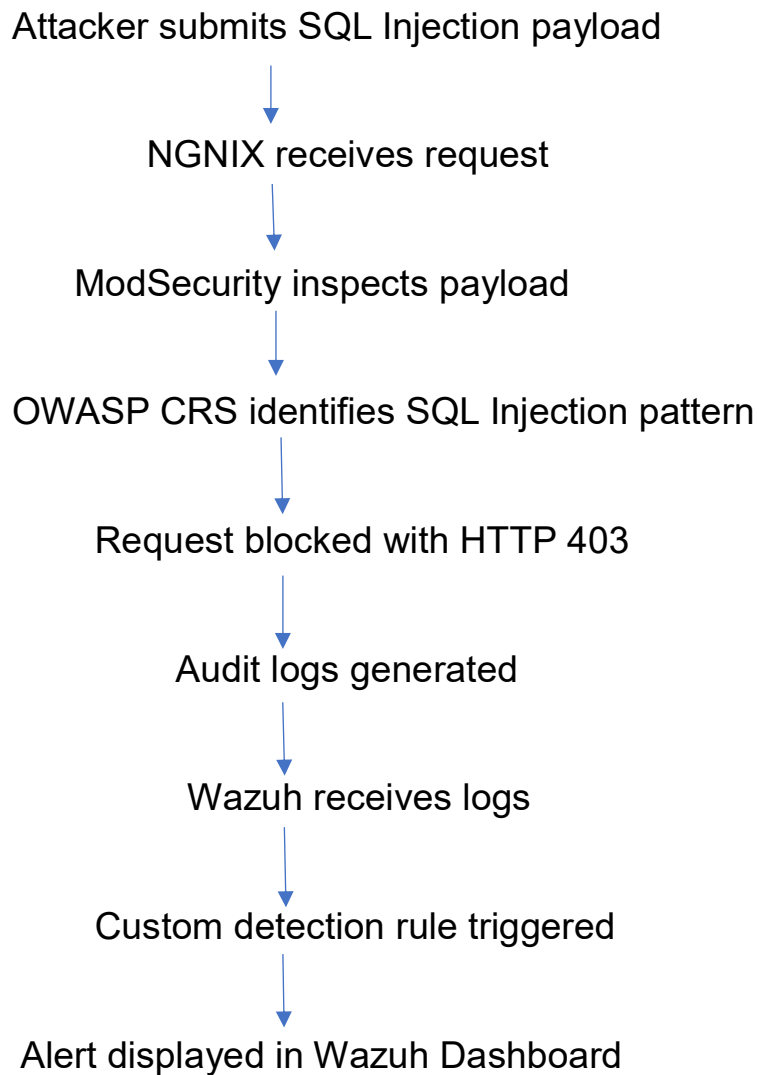
Detection Component: ModSecurity WAF

SIEM Detection: Wazuh Custom Rule (100501)

Severity: Level 15

Action Taken: Request blocked (HTTP 403)

SQL Injection Workflow



7.3 Remote Code Execution (RCE)

Remote Code Execution (RCE) attacks allow an attacker to execute arbitrary commands

on a target system. These attacks represent one of the highest-risk web application

vulnerabilities due to their potential impact.

Attack Scenario

The DVWA Command Execution module was used to simulate RCE attacks.

Sample payload

127.0.0.1 && whoami

Detection Process

1. The malicious command was submitted through DVWA.
2. ModSecurity detected suspicious command execution patterns.
3. Custom Wazuh rules classified the event as an RCE attack.
4. Security alerts were generated and mapped to MITRE ATT&CK techniques.
5. Active Response mechanisms were triggered when configured conditions were met.

Results

The attack was successfully detected and classified as a Remote Code Execution attempt.

SOCShield generated alerts, MITRE ATT&CK mappings, and automated response actions where applicable.



```

--nIC8FYwL...F--
HTTP/1.1 403
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 06 Jun 2026 22:07:53 GMT
Content-Length: 162
Content-Type: text/html
Connection: keep-alive

--nIC8FYwL...H--
ModSecurity: Warning, Matched "Operator 'Rx' with parameter '{(?:[!d-;+][![[d-f;+]]|[[d-f;+]](?:[d+;}]$)' against variable 'REQUEST_HEADERS:Host' (Value: '192.168.100.1') [file '/etc/modsecurity-crs/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf'] [line '756'] [id '920350'] [rev ''] [msg 'Host header is a numeric IP address'] [data '192.168.100.1'] [severity '4'] [ver 'OWASP CRS/4.26.0-dev'] [maturity '0'] [accuracy '0'] [tag 'application-multi'] [tag 'language-multi'] [tag 'platform-multi'] [tag 'attack-protocol'] [tag 'paranoia-level/1'] [tag 'OWASP_CRS'] [tag 'OWASP_CRS/PROTOCOL-ENFORCEMENT'] [tag 'capec/1000/152/248/88'] [hostname '192.168.100.1'] [url '/DVWA/vulnerabilities/exec/'] [unique_id '17807836739.749053'] [ref 'o0,1300,13v40,13']
ModSecurity: Warning, Matched "Operator 'Rx' with parameter '{(?:[!b|~|\\|\\x5c]*(?:(?:[!\\|\\|&]|\\s|x0b)*)?\\s!#([\\^\\-0-9!@_a-\\[\\]]*\\x5c?|\\|\\|\\|\\x5c)*(?:(?:[!\\|\\|&]|\\s|x0b)*)? (6011 characters omitted)' against variable 'ARGS_NAMES:whoami' (Value: 'whoami') [file '/etc/modsecurity-crs/rules/REQUEST-932-APPLICATION-ATTACK-RCE.conf'] [line '567'] [id '932260'] [rev ''] [msg 'Remote Command Execution: Direct Unix Command Execution'] [data 'Matched Data: whoami found within ARGS_NAMES: whoami: whoami'] [severity '2'] [ver 'OWASP CRS/4.26.0-dev'] [maturity '0'] [accuracy '0'] [tag 'application-multi'] [tag 'language-shell'] [tag 'platform-unix'] [tag 'attack-rce'] [tag 'paranoia-level/1'] [tag 'OWASP_CRS'] [tag 'OWASP_CRS/ATTACK-RCE'] [tag 'capec/1000/152/248/88'] [hostname '192.168.100.1'] [url '/DVWA/vulnerabilities/exec/'] [unique_id '17807836739.749053'] [ref 'o0,7v588,7']
ModSecurity: Access denied with code 403 (phase 2). Matched "Operator 'Ge' with parameter '5' against variable 'TX:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: '8') [file '/etc/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf'] [line '222'] [id '949110'] [rev ''] [msg 'Inbound Anomaly Score Exceeded (Total Score: 8)'] [data ''] [severity '0'] [ver 'OWASP CRS/4.26.0-dev'] [maturity '0'] [accuracy '0'] [tag 'anomaly-evaluation'] [tag 'OWASP_CRS'] [hostname '192.168.100.1'] [url '/DVWA/vulnerabilities/exec/'] [unique_id '17807836739.749053'] [ref '']

```

Figure 7.8: ModSecurity audit logs showing detection of a Remote Code Execution attempt through OWASP CRS RCE detection rules.

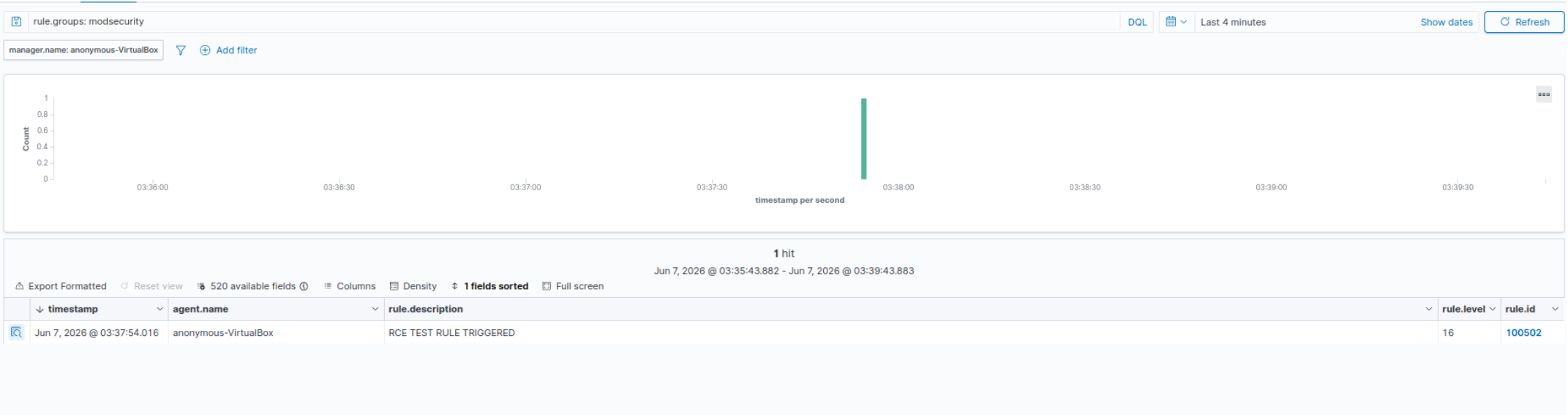


Figure 7.9: Wazuh SIEM generated a high-severity alert after receiving and processing the ModSecurity RCE detection event.

8. MITRE ATT&CK Mapping

8.1 Overview

The MITRE ATT&CK Framework is a globally recognized knowledge base of adversary tactics and techniques derived from real-world cyberattack observations. It provides a standardized method for classifying and understanding attacker behavior throughout the attack lifecycle.

SOCShield integrates MITRE ATT&CK mapping within the detection workflow to provide additional context for security analysts. By associating detected attacks with relevant ATT&CK techniques, analysts can better understand attacker objectives, improve threat hunting activities, and accelerate incident response.

The integration of MITRE ATT&CK mapping enhances the overall effectiveness of the Security Operations Center (SOC) by transforming raw security alerts into actionable threat intelligence.

8.2 MITRE ATT&CK Integration in SOCShield

When ModSecurity detects a malicious request and Wazuh generates a corresponding

alert, the event is enriched with MITRE ATT&CK tactics and techniques through custom

detection rules.

This process provides:

- Standardized attack classification
- Improved analyst visibility
- Enhanced incident investigation
- Threat hunting support
- Better understanding of adversary behavior

The mapped ATT&CK techniques are displayed directly within Wazuh alerts and dashboards.

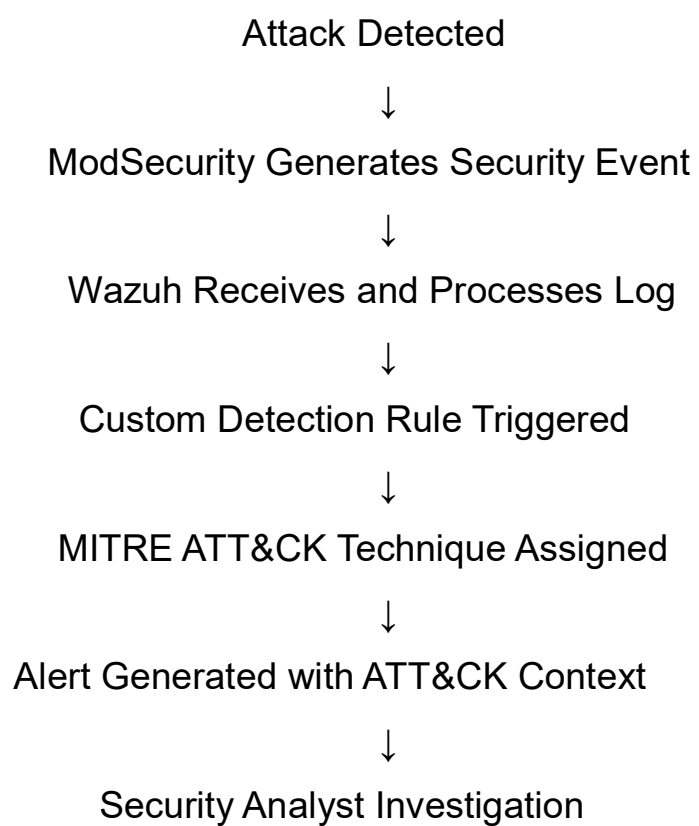
8.3 Attack Mapping Examples

The following table illustrates the MITRE ATT&CK techniques associated with attacks detected during the SOCSshield implementation.

Attack Type	MITRE Technique ID	Technique Name
Tactic		
Cross-Site Scrip- Execution	T1059	Command and Scripting
ting (XSS)		Interpreter
Remote Code Execution	T1059.004/	Command and Scripting
Execution	T1059	Interpreter
Directory Discovery Discovery Discovery	T1083	File and Directory
Events		
Web Application Initial	T1190	Exploit Public-Facing
Attacks		Application
Access		

8.4 MITRE ATT&CK Workflow

The attack classification workflow implemented in SOCSHield is shown below:



This workflow enables security analysts to quickly identify attacker objectives and understand the significance of security alerts.

8.5 Benefits of MITRE ATT&CK Mapping

The integration of MITRE ATT&CK within SOCSshield provides several operational advantages:

Standardized Threat Classification

Security events are mapped to a globally recognized framework, ensuring consistent threat categorization.

Improved Threat Hunting

Analysts can search for ATT&CK techniques across historical security events and identify patterns of malicious activity.

Enhanced Incident Investigation

Mapped tactics and techniques provide additional context that assists analysts during incident response activities.

Security Monitoring Optimization

ATT&CK mappings help prioritize alerts based on attacker behavior and attack objectives.

SOC Maturity Improvement

The use of ATT&CK-based detections aligns SOCSshield with modern Security Operations Center methodologies and industry best practices.

8.6 Results

The implementation successfully demonstrated the integration of MITRE ATT&CK mappings within the attack detection workflow. Security events generated from XSS, SQL Injection, Remote Code Execution, and discovery-related activities were enriched with ATT&CK context, providing analysts with improved visibility into adversary behavior and attack progression.

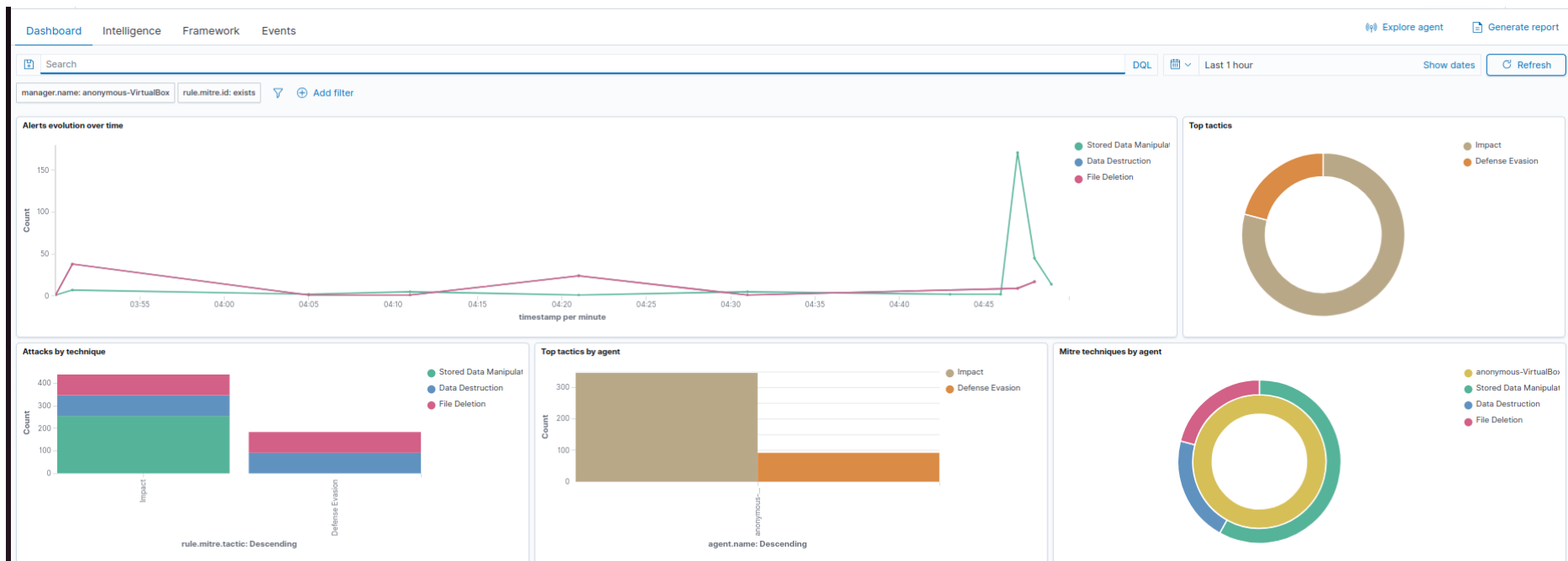


Figure 8.1: MITRE ATT&CK Dashboard displaying mapped tactics and techniques associated with monitored security events.

MITRE ATT&CK

API default a

363 hits

Jun 7, 2026 @ 03:50:54.735 - Jun 7, 2026 @ 04:50:54.735

Export FormattedReset view520 available fieldsColumnsDensity1 fields sortedExit full screen

timestamp	agent.name	rule.mitre.id	rule.mitre.tactic	rule.description	rule.level	rule.id
Jun 7, 2026 @ 04:50:46.482	anonymous-VirtualBox	T1070.004 T1	Defense Evasion, Impact	File deleted.	7	553
Jun 7, 2026 @ 04:50:46.481	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:44.465	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:34.499	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:29.214	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:29.207	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:21.820	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:19.976	anonymous-VirtualBox	T1070.004 T1	Defense Evasion, Impact	File deleted.	7	553
Jun 7, 2026 @ 04:50:19.660	anonymous-VirtualBox	T1070.004 T1	Defense Evasion, Impact	File deleted.	7	553
Jun 7, 2026 @ 04:50:19.285	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:14.414	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:14.408	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550
Jun 7, 2026 @ 04:50:08.718	anonymous-VirtualBox	T1070.004 T1	Defense Evasion, Impact	File deleted.	7	553
Jun 7, 2026 @ 04:50:08.715	anonymous-VirtualBox	T1070.004 T1	Defense Evasion, Impact	File deleted.	7	553
Jun 7, 2026 @ 04:50:08.709	anonymous-VirtualBox	T1565.001	Impact	Integrity checksum changed.	7	550

Figure 8.2: Individual security events enriched with MITRE ATT&CK technique and tactic information.

- SOCShield successfully leveraged the MITRE ATT&CK framework to enrich security events with adversary tactics and techniques. This enabled standardized threat classification, improved analyst visibility, and enhanced incident investigation capabilities.

9. Active Response

9.1 Overview

Detecting malicious activity is only the first step in the incident response lifecycle. To minimize the impact of security incidents, organizations must be capable of responding quickly to identified threats. Manual response processes often introduce delays that allow attackers additional time to continue malicious activities within the environment.

To address this challenge, SOCSshield implements an automated Active Response mechanism using Wazuh's firewall-drop capability. When predefined security conditions are met, the platform automatically executes response actions to block malicious source IP addresses, reducing the attack surface and limiting further malicious activity.

The Active Response functionality enables SOCSshield to transition from a passive monitoring platform to an intelligent security solution capable of taking immediate defensive actions against identified threats.

9.2 Active Response Architecture

The Active Response workflow consists of the following components:

1. ModSecurity detects malicious web requests.
2. Wazuh analyzes and correlates security events.
3. Custom detection rules identify high-severity attacks.
4. Active Response is triggered based on predefined rule conditions.
5. The firewall-drop script is executed automatically.
6. The malicious source IP address is blocked at the host firewall.
7. An alert is generated and recorded within the Wazuh Dashboard.

The automated workflow significantly reduces the time required to contain malicious activity.

9.3 Active Response Configuration

Wazuh Active Response was configured using the built-in firewall-drop script. The response

Mechanism was linked to custom web attack detection rules developed for SOCSshield.

The Active Response configuration was designed to:

- Identify malicious source IP addresses.
- Automatically block attacker systems.
- Generate response-related alerts.
- Log all response actions for auditing and investigation purposes.

The firewall-drop mechanism operates at the operating system level and dynamically updates firewall rules whenever a qualifying attack is detected.

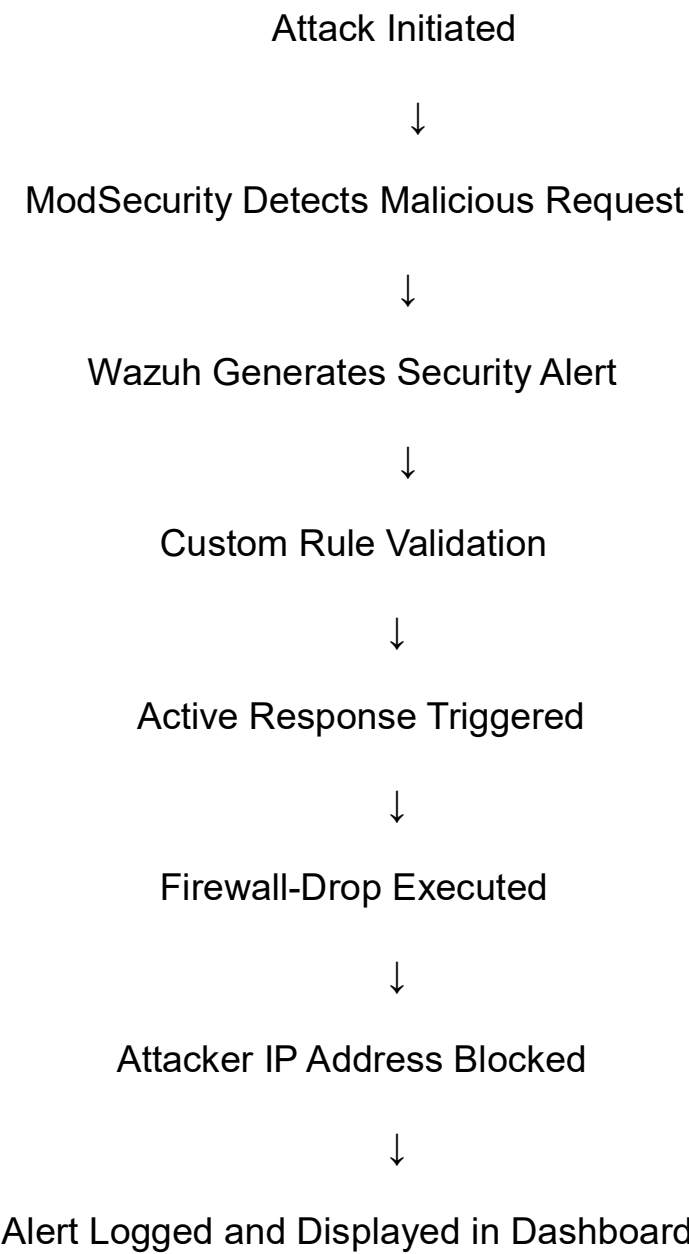
```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100500,100501,100502</rules_id>
  <timeout>600</timeout>
</active-response>
```

Figure 9.1 – Wazuh Active Response Configuration Using Firewall-Drop

Figure 9.1 illustrates the Active Response configuration implemented within Wazuh. When security events matching custom rule IDs 100500, 100501, and 100502 are detected, the firewall-drop response is automatically executed. The malicious source IP address is blocked locally for a period of 600 seconds, helping to contain potentially harmful activities and reduce the risk of repeated attack attempts.

9.4 Attack Response Workflow

The operational workflow implemented within SOCSshield is shown below:

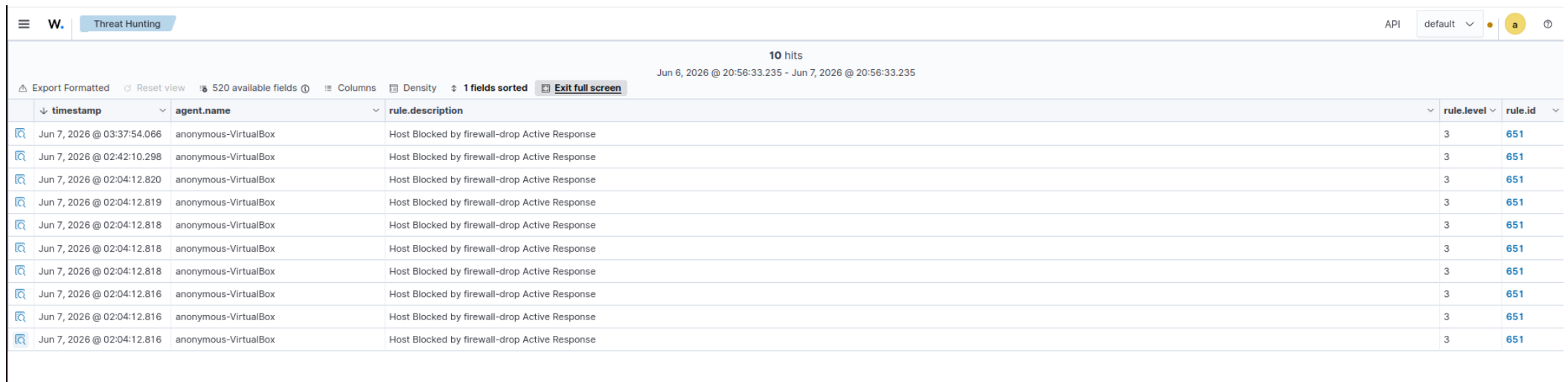


9.5 Validation and Testing

To validate the Active Response capability, multiple attack scenarios were executed against the DVWA web application using Kali Linux and Burp Suite. High-severity attacks such as Remote Code Execution (RCE) were successfully detected by ModSecurity and forwarded to Wazuh for analysis.

Upon detection, the custom Wazuh rule triggered the firewall-drop Active Response action. The attacker source IP address was automatically added to the firewall block list, preventing additional requests from reaching the protected web application.

The generated alerts confirmed that the response action had been executed successfully and that the malicious IP address had been blocked.



The screenshot displays the Wazuh Threat Hunting interface. At the top, there's a navigation bar with a menu icon, the 'W.' logo, and a 'Threat Hunting' tab. On the right, there are controls for 'API', 'default' view, and a user profile icon labeled 'a'. Below the navigation bar, a summary bar indicates '10 hits' for the time range 'Jun 6, 2026 @ 20:56:33.235 - Jun 7, 2026 @ 20:56:33.235'. Below this, a toolbar offers options like 'Export Formatted', 'Reset view', '520 available fields', 'Columns', 'Density', '1 fields sorted', and an 'Exit full screen' button. The main area contains a table with the following columns: 'timestamp', 'agent.name', 'rule.description', 'rule.level', and 'rule.id'. The table lists 10 identical alerts, all from 'anonymous-VirtualBox' agents, with the description 'Host Blocked by firewall-drop Active Response' and rule ID '651'.

timestamp	agent.name	rule.description	rule.level	rule.id
Jun 7, 2026 @ 03:37:54.066	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:42:10.298	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.820	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.819	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.818	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.818	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.818	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.818	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.816	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651
Jun 7, 2026 @ 02:04:12.816	anonymous-VirtualBox	Host Blocked by firewall-drop Active Response	3	651

Figure 9.2 – Firewall-Drop Alert Generated by Wazuh

```

2026/06/07 02:04:10 active-response/bin/firewall-drop: Cannot read 'srcip' from data
2026/06/07 02:42:09 active-response/bin/firewall-drop: Starting
2026/06/07 02:42:09 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":[],"alert":{"timestamp":"2026-06-07T02:42:09.716+0530","rule":{"level":15,"description":"Modsecurity SQL Injection Attack Detected","id":"100501","mitre":{"id":["T1190"],"tactic":["Initial Access"],"technique":["Exploit Public-Facing Application"]},"firedtimes":1,"mail":true,"groups":["modsecurity","web_attacks","sql","web_attack","modsecurity"]},"agent":{"id":"000","name":"anonymous-VirtualBox"},"manager":{"name":"anonymous-VirtualBox"},"id":"1780780329.911074"},"full_log":"ModSecurity: Warning. detected SQLi using libinjection. [file \"/etc/modsecurity-crs/rules/REQUEST-942-APPLICATION-ATTACK-SQLI.conf"] [line \"/46"] [id \"/942100"] [rev \"/"] [msg \"/SQL Injection Attack Detected via libinjection"] [data \"/Matched Data: sos found within ARGS:id : %OR%1='1'"] [severity \"/2"] [ver \"/OWASP_CRS/4.26.0-dev"] [maturity \"/0"] [accuracy \"/0"] [tag \"/application-multi"] [tag \"/language-multi"] [tag \"/platform-multi"] [tag \"/attack-sqli"] [tag \"/paranoia-level/1"] [tag \"/OWASP_CRS"] [tag \"/OWASP_CRS/ATTACK-SQLI"] [tag \"/capec/1000/152/248/66"] [hostname \"/192.168.100.1"] [uri \"/DVWA/vulnerabilities/sqli/"] [unique_id \"/178078032952.350572"] [ref \"/v35,11"]},"decoder":{"name":"modsec-cus tom-ip"},"location":"/var/log/modsec_audit.log"},"program":"active-response/bin/firewall-drop"}}

2026/06/07 02:42:09 active-response/bin/firewall-drop: Cannot read 'srcip' from data
2026/06/07 03:37:54 active-response/bin/firewall-drop: Starting
2026/06/07 03:37:54 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":[],"alert":{"timestamp":"2026-06-07T03:37:54.016+0530","rule":{"level":16,"description":"RCE TEST RULE TRIGGERED","id":"100502","mitre":{"id":["T1059.008"],"tactic":["Execution"],"technique":["Network Device CLI"]},"firedtimes":1,"mail":true,"groups":["modsecurity","web_attacks","web","rce","attack","modsecurity"]},"agent":{"id":"000","name":"anonymous-VirtualBox"},"manager":{"name":"anonymous-VirtualBox"},"id":"1780783674.1930053"},"full_log":"2026/06/07 03:37:53 [error] 1908#1908: *37 [client 192.168.100.20] ModSecurity: Access denied with code 403 (phase 2). Matched \"/Operator 'Ge' with parameter 'S' against variable 'TX:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: '8' ) [file \"/etc/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line \"/222"] [id \"/949110"] [rev \"/"] [msg \"/Inbound Anomaly Score Exceeded (Total Score: 8)"] [data \"/"] [severity \"/0"] [ver \"/OWASP_CRS/4.26.0-dev"] [maturity \"/0"] [accuracy \"/0"] [tag \"/anomaly-evaluation"] [tag \"/OWASP_CRS"] [hostname \"/192.168.100.1"] [uri \"/DVWA/vulnerabilities/exec/"] [unique_id \"/17807836739.749053"] [ref \"/"], client: 192.168.100.20, server: , request: \"/POST /DVWA/vulnerabilities/exec/ HTTP/1.1", host: \"/192.168.100.1", referer: \"/http://192.168.100.1/DVWA/vulnerabilities/exec/"/,"decoder":{"parent":"nginx-errorlog","name":"nginx-errorlog"},"data":{"srcip":"192.168.100.20"},"location":"/var/log/nginx/error.log"},"program":"active-response/bin/firewall-drop"}}

2026/06/07 03:37:54 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"firewall-drop","module":"active-response"},"command":"check_keys","parameters":{"keys":["192.168.100.20"]}}
2026/06/07 03:37:54 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"continue","parameters":{"extra_args":[],"alert":{"timestamp":"2026-06-07T03:37:54.016+0530","rule":{"level":16,"description":"RCE TEST RULE TRIGGERED","id":"100502","mitre":{"id":["T1059.008"],"tactic":["Execution"],"technique":["Network Device CLI"]},"firedtimes":1,"mail":true,"groups":["modsecurity","web_attacks","web","rce","attack","modsecurity"]},"agent":{"id":"000","name":"anonymous-VirtualBox"},"manager":{"name":"anonymous-VirtualBox"},"id":"1780783674.1930053"},"full_log":"2026/06/07 03:37:53 [error] 1908#1908: *37 [client 192.168.100.20] ModSecurity: Access denied with code 403 (phase 2). Matched \"/Operator 'Ge' with parameter 'S' against variable 'TX:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: '8' ) [file \"/etc/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line \"/222"] [id \"/949110"] [rev \"/"] [msg \"/Inbound Anomaly Score Exceeded (Total Score: 8)"] [data \"/"] [severity \"/0"] [ver \"/OWASP_CRS/4.26.0-dev"] [maturity \"/0"] [accuracy \"/0"] [tag \"/anomaly-evaluation"] [tag \"/OWASP_CRS"] [hostname \"/192.168.100.1"] [uri \"/DVWA/vulnerabilities/exec/"] [unique_id \"/17807836739.749053"] [ref \"/"], client: 192.168.100.20, server: , request: \"/POST /DVWA/vulnerabilities/exec/ HTTP/1.1", host: \"/192.168.100.1", referer: \"/http://192.168.100.1/DVWA/vulnerabilities/exec/"/,"decoder":{"parent":"nginx-errorlog","name":"nginx-errorlog"},"data":{"srcip":"192.168.100.20"},"location":"/var/log/nginx/error.log"},"program":"active-response/bin/firewall-drop"}}

2026/06/07 03:37:54 active-response/bin/firewall-drop: Ended
2026/06/07 03:47:55 active-response/bin/firewall-drop: Starting
2026/06/07 03:47:55 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"delete","parameters":{"extra_args":[],"alert":{"timestamp":"2026-06-07T03:37:54.016+0530","rule":{"level":16,"description":"RCE TEST RULE TRIGGERED","id":"100502","mitre":{"id":["T1059.008"],"tactic":["Execution"],"technique":["Network Device CLI"]},"firedtimes":1,"mail":true,"groups":["modsecurity","web_attacks","web","rce","attack","modsecurity"]},"agent":{"id":"000","name":"anonymous-VirtualBox"},"manager":{"name":"anonymous-VirtualBox"},"id":"1780783674.1930053"},"full_log":"2026/06/07 03:37:53 [error] 1908#1908: *37 [client 192.168.100.20] ModSecurity: Access denied with code 403 (phase 2). Matched \"/Operator 'Ge' with parameter 'S' against variable 'TX:BLOCKING_INBOUND_ANOMALY_SCORE' (Value: '8' ) [file \"/etc/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line \"/222"] [id \"/949110"] [rev \"/"] [msg \"/Inbound Anomaly Score Exceeded (Total Score: 8)"] [data \"/"] [severity \"/0"] [ver \"/OWASP_CRS/4.26.0-dev"] [maturity \"/0"] [accuracy \"/0"] [tag \"/anomaly-evaluation"] [tag \"/OWASP_CRS"] [hostname \"/192.168.100.1"] [uri \"/DVWA/vulnerabilities/exec/"] [unique_id \"/17807836739.749053"] [ref \"/"], client: 192.168.100.20, server: , request: \"/POST /DVWA/vulnerabilities/exec/ HTTP/1.1", host: \"/192.168.100.1", referer: \"/http://192.168.100.1/DVWA/vulnerabilities/exec/"/,"decoder":{"parent":"nginx-errorlog","name":"nginx-errorlog"},"data":{"srcip":"192.168.100.20"},"location":"/var/log/nginx/error.log"},"program":"active-response/bin/firewall-drop"}}

```

Figure 9.3 – Attacker Source IP Blocked Successfully

10. Threat Intelligence Integration

10.1 Overview

Threat Intelligence plays a critical role in modern Security Operations Center (SOCs) by providing contextual information about indicators of compromise (IOCs), malicious files, domains, URLs, and IP addresses. Integrating threat intelligence into the detection workflow helps security analysts make informed decisions and prioritize security incidents more effectively.

SOCShield integrates VirusTotal Threat Intelligence with Wazuh SIEM to enrich security alerts generated from web application attacks. VirusTotal aggregates threat intelligence from multiple security vendors and provides reputation information for suspicious files, domains, URLs, and IP addresses.

By incorporating VirusTotal into the detection workflow, SOCShield enhances alert visibility and enables analysts to quickly determine whether a detected indicator is known to be malicious, suspicious.

10.2 Integration Architecture

The VirusTotal integration was implemented using the native Wazuh Integration Module. Security events generated by Wazuh are forwarded to VirusTotal for reputation analysis and threat intelligence enrichment.

The integration workflow consists of the following steps:

1. Wazuh detects a suspicious event.
2. Relevant indicators are extracted from the alert.
3. The VirusTotal integration module submits the indicators to VirusTotal.
4. VirusTotal performs reputation analysis.
5. Reputation results are returned to Wazuh.
6. The enriched alert is displayed within the Wazuh Dashboard.

This process provides additional context for security analyst during threat investigations.

10.3 VirusTotal Configuration

VirusTotal integration was configured within the Wazuh Manager using the official VirusTotal API. The integration required the generation of an API key and modification of the Wazuh configuration file to enable communication with VirusTotal services.

The configuration included:

- VirusTotal API Key Configuration
- Wazuh Integration Module Setup
- Alert Enrichment Configuration
- Integration Validation and Testing

After successful configuration, Wazuh was capable of querying VirusTotal and incorporating reputation information into generation alerts.

```
<integration>
  <name>virustotal</name>
  <api_key>xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx</api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>
```

Figure 10.1 – VirusTotal Integration Configuration in Wazuh Manager

10.4 Threat Intelligence Enrichment Workflow

When a security event is generated, SOCSshield attempts to enrich the event using VirusTotal reputation data. This enrichment process provides additional intelligence that assists analysts during investigations.

The enrichment workflow includes:

- Indicator extraction
- Reputation lookup
- Threat scoring
- Alert enhancement
- Analyst review

By combining local security telemetry with external threat intelligence sources, SOCSshield improves overall situational awareness and accelerates incident response activities.

10.5 Validation and Testing

The VirusTotal integration was validated by generating security events and monitoring the resulting enrichment alerts within Wazuh. Multiple indicators were submitted for analysis to verify communication between Wazuh and VirusTotal.

Testing confirmed that the integration successfully communicated with the VirusTotal platform and generated enrichment-related events within the Wazuh Dashboard.

During testing, public API limitations were encountered due to request rate restrictions imposed by VirusTotal. These limitations were expected and demonstrated the operational constraints associated with public API usage.

W.

Threat Hunting

API default a

5,209 hits

Jun 6, 2026 @ 22:03:28.560 - Jun 7, 2026 @ 22:03:28.560

Export Formatted

Reset view

520 available fields

Columns

Density

1 fields sorted

Exit full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Jun 7, 2026 @ 22:03:25.535	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:24.139	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:22.908	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:21.696	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:20.619	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:19.435	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:19.102	anonymous-VirtualBox	Integrity checksum changed.	7	550
Jun 7, 2026 @ 22:03:17.955	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:16.622	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:15.189	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:13.896	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:12.617	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:11.331	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:10.200	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101
Jun 7, 2026 @ 22:03:09.033	anonymous-VirtualBox	VirusTotal: Error: Public API request rate limit reached	3	87101

Figure 10.2 – VirusTotal API Rate Limitation Encountered During Testing

468 hits				
Jun 6, 2026 @ 22:09:01.655 - Jun 7, 2026 @ 22:09:01.655				
Export Formatted Reset view 520 available fields Columns Density 1 fields sorted Exit full screen				
timestamp	agent.name	rule.description	rule.level	rule.id
Jun 7, 2026 @ 22:08:21.743	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:08:11.029	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:08:09.766	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:07:03.034	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:07:01.805	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:07:00.469	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:06:59.215	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:06:05.316	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:06:02.974	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:06:00.982	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:05:59.450	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:05:04.237	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:05:02.903	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:05:01.163	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jun 7, 2026 @ 22:04:59.774	anonymous-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103

Figure 10.3 – VirusTotal Threat Intelligence Enrichment Result

10.6 Benefits of Threat Intelligence Integration

The integration of VirusTotal within SOCShield provides several operational benefits:

- Enhanced threat visibility
- Reputation-based analysis
- Improved incident investigation
- Faster analyst decision-making
- External threat intelligence enrichment
- Better threat prioritization

Threat intelligence enrichment allows analysts to correlate internal security events with external intelligence sources, improving the overall effectiveness of the Security Operations Center.

11. Email Alerting

11.1 Overview

Timely notification of security incidents is essential for effective incident response. While dashboards provide centralized visibility into security events, analysts may not continuously monitor the Security Operations Center (SOC) console. As a result, critical alerts can be overlooked or delayed.

To address this challenge, SOCSshield implements a real-time email alerting mechanism using Wazuh, Postfix Mail Transfer Agent (MTA), and Gmail SMTP services. The alerting system automatically notifies security analysts whenever predefined security events are detected, ensuring rapid awareness and response to potential threats.

The email alerting capability enhances the effectiveness of SOCSshield by providing immediate notification of security incidents without requiring constant dashboard monitoring.

11.2 Email Alerting Architecture

The email notification workflow consists of the following components:

1. ModSecurity detects a malicious web request.
2. Wazuh generates a security alert.
3. Alert severity and rule conditions are evaluated.
4. Wazuh forwards the alert to Postfix.
5. Postfix relays the email through Gmail SMTP
6. The security analyst receives a notification email.

This workflow ensures that important security events are communicated to analysts as soon as they occur.

11.3 Email Alert Configuration

Email alerting was configured within the Wazuh Manager using Postfix as the Mail Transfer Agent (MTA). Gmail SMTP services were used to relay outgoing email notifications.

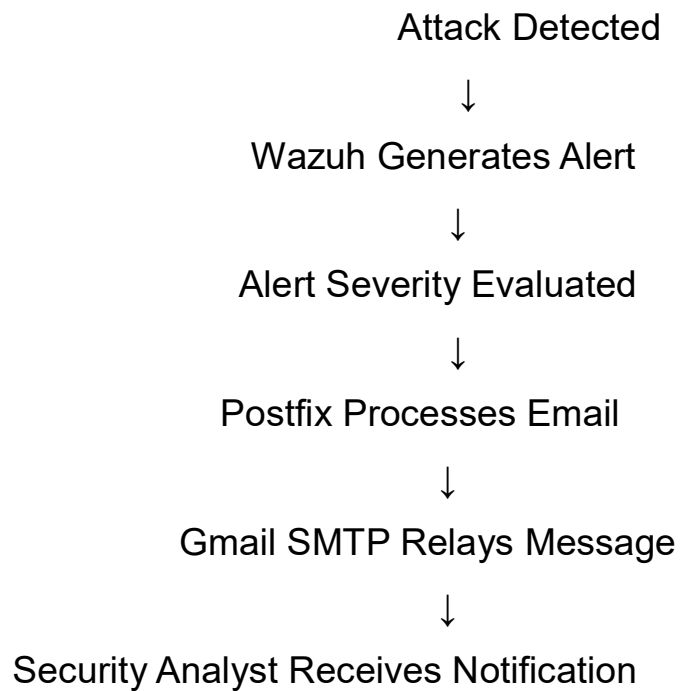
The configuration process included:

- Postfix installation and configuration
- Gmail SMTP integration
- Authentication using App Passwords
- Wazuh email notification configuration
- Alert delivery validation and testing

After successful configuration, Wazuh was capable of automatically generating and sending email notifications for security events.

11.4 Alert Notification Workflow

The operational workflow implemented within SOCSHield is shown below:



This automated process reduces the time required for analysts to become aware of security incidents.

11.5 Validation and Testing

The email alerting functionality was validated by generating security events through simulated attack scenarios against the DVWA web application.

Multiple attacks, including Cross-Site Scripting (XSS), SQL Injection (SQLi), and Remote Code Execution (RCE), were executed to verify alert generation and email delivery functionality.

Testing confirmed that security alerts generated within Wazuh were successfully processed by Postfix and delivered through Gmail SMTP. Notification emails were received by the designated analyst mailbox, demonstrating successful end-to-end alert delivery.

The testing process also included troubleshooting SMTP authentication and Gmail application password configuration to ensure reliable email transmission.

11.6 Benefits of Email Alerting

The implementation of email alerting within SOCSshield provides several operational advantages:

- Immediate analyst notification
- Faster incident response
- Improved threat visibility
- Reduced dependency on dashboard monitoring
- Automated alert delivery
- Enhanced security operations efficiency

By integrating automated email notifications into the SOC workflow, SOCSshield improves analyst awareness and supports timely response to security incidents.


```
<email_notification>yes</email_notification>  
<smtp_server>localhost</smtp_server>  
<email_from>xxxxxxx@gmail.com</email_from>  
<email_to>xxxxxxx@gmail.com</email_to>  
<email_maxperhour>20</email_maxperhour>
```

Figure 11.1 – Wazuh Email Notification Configuration

Figure 11.1 shows the email notification configuration implemented within the Wazuh Manager. Email alerting was enabled by configuring the SMTP server, sender address, recipient address, and notification limits. This configuration allows Wazuh to automatically generate and forward security alerts to the designated analyst mailbox.

```
relayhost = [smtp.gmail.com]:587
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options = noanonymous
smtp_tls_security_level = encrypt
smtp_tls_CAfile = /etc/ssl/certs/ca-certificates.crt
```

Figure 11.2 – Postfix Gmail SMTP Relay Configuration

Figure 11.2 shows the Postfix SMTP relay configuration used within SOCSshield. Postfix was configured to authenticate with Gmail SMTP services using TLS encryption and application-password-based authentication, enabling secure delivery of security alert notifications generated by Wazuh.

Wazuh notification - anonymous-VirtualBox - Alert level 16 Inbox x

 Convert to PDF  

F

Wazuh 
to me ▾

03:38 (19 hours ago) ☆ 😊 ↶ ⋮

Wazuh Notification.
2026 Jun 07 03:37:54

Received From: anonymous-VirtualBox->/var/log/nginx/error.log
Rule: 100502 fired (level 16) -> "RCE TEST RULE TRIGGERED"
Src IP: 192.168.100.20
Portion of the log(s):

2026/06/07 03:37:53 [error] 1908#1908: *37 [client 192.168.100.20] ModSecurity: Access denied with code 403 (phase 2). Matched "Operator `Ge' with parameter `5' against variable `TX:BLOCKING\_INBOUND\_ANOMALY\_SCORE' (Value: `8') [file "/etc/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "222"] [id "949110"] [rev ""] [msg "Inbound Anomaly Score Exceeded (Total Score: 8)"] [data ""] [severity "0"] [ver "OWASP_CRS/4.26.0-dev"] [maturity "0"] [accuracy "0"] [tag "anomaly-evaluation"] [tag "OWASP_CRS"] [hostname "192.168.100.1"] [uri "/DVWA/vulnerabilities/exec/"] [unique_id "17807836739.749053"] [ref ""], client: 192.168.100.20, server: , request: "POST /DVWA/vulnerabilities/exec/ HTTP/1.1", host: "192.168.100.1", referer: "<http://192.168.100.1/DVWA/vulnerabilities/exec/>"

--END OF NOTIFICATION

Figure 11.3 – Security Alert Email Received by Analyst Following Detection of a Remote Code Execution (RCE) Attack

12. Dashboard & Monitoring

12.1 Overview

Modern Security Operations Centers (SOCs) rely heavily on dashboards to provide real-time visibility into security events, threat trends, and incident response activities. Effective dashboards enable analysts to quickly identify suspicious activities, prioritize investigations, and monitor security operations from a centralized interface.

SOCShield incorporates multiple custom dashboards within Wazuh to visualize web attack activity, monitor active response actions, analyze attacker behavior, and support threat hunting operations. These dashboards transform raw security events into actionable intelligence, improving situational awareness and accelerating incident response

12.2 Web Attack Monitoring Dashboard

The Web Attack Monitoring Dashboard was developed to provide a consolidated view of web application attacks detected by ModSecurity and analysed by Wazuh.

The dashboard includes:

- Attack Timeline Visualization
- Attack Type Distribution
- Severity Distribution Analysis
- Web Attack Event Table

The dashboard enables security analysts to monitor attack trends and identify the most frequently occurring attack types within the protected environment.

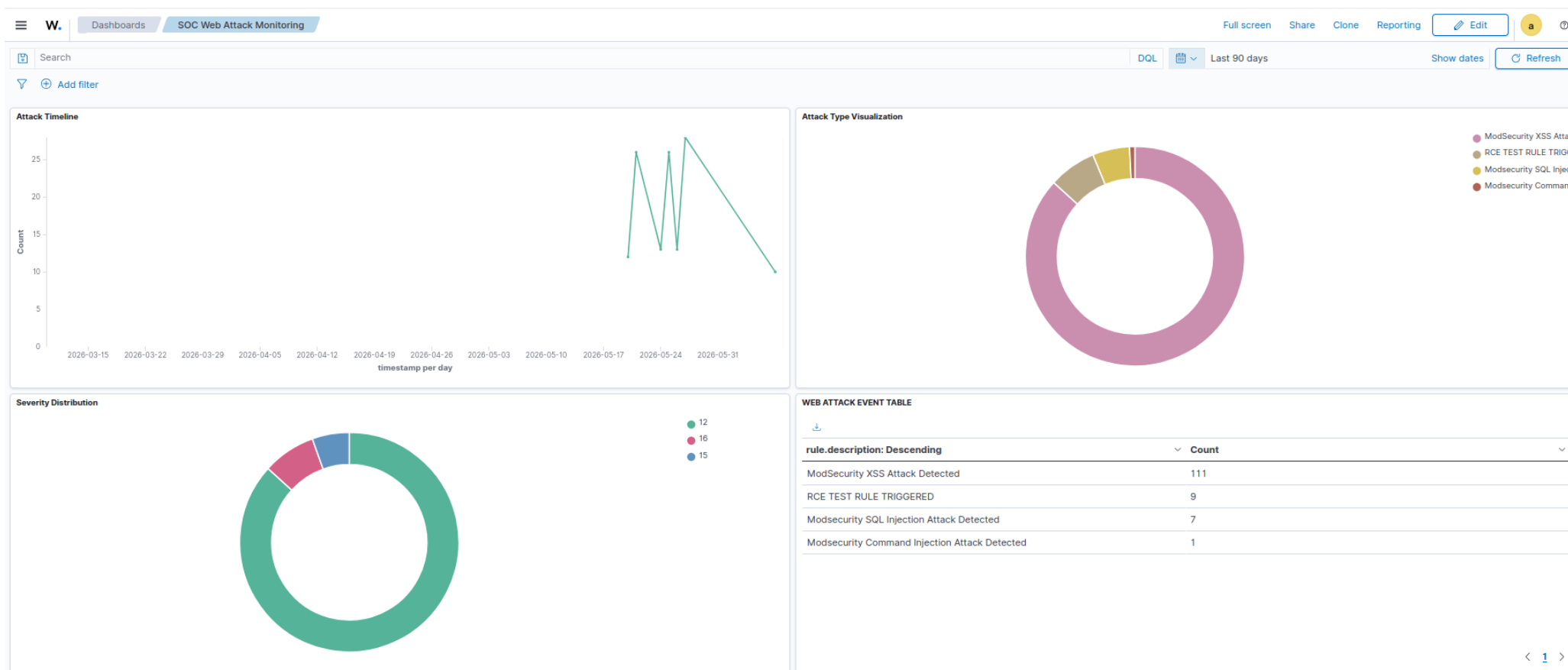


Figure 12.1 – SOC Web Attack Monitoring Dashboard

Figure 12.1 presents the SOC Web Attack Monitoring Dashboard developed within SOCSshield. The dashboard provides real-time visibility into web application attacks detected by ModSecurity and analyzed by Wazuh. It includes attack timelines, attack type distribution, severity distribution, and a summarized event table. During testing, the dashboard successfully identified Cross-Site Scripting (XSS), SQL Injection (SQLi), Remote Code Execution (RCE), and Command Injection attacks.

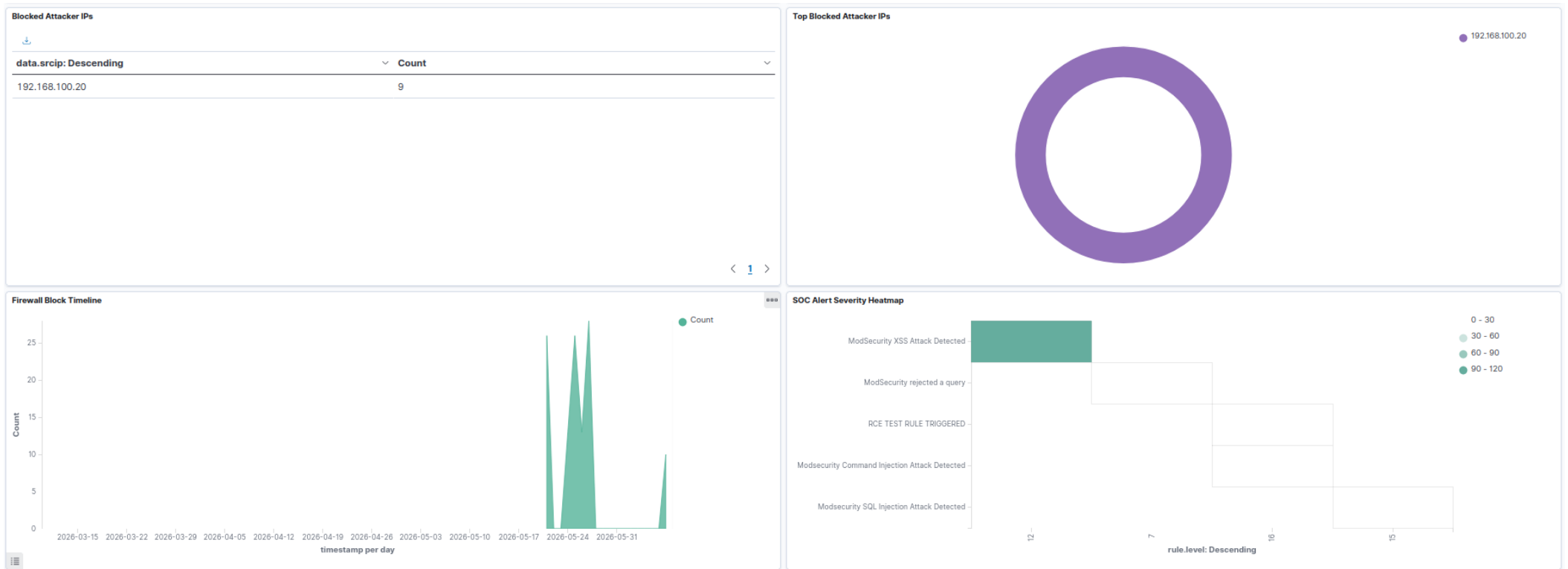


Figure 12.2 – Active Response Monitoring Dashboard

Figure 12.2 illustrates the Active Response Monitoring Dashboard. The dashboard provides visibility into blocked attacker IP addresses, firewall-drop activity, and response timelines. It confirms the successful execution of Wazuh Active Response actions and demonstrates automated containment of malicious activity by blocking attacker IP addresses following high-severity security events.

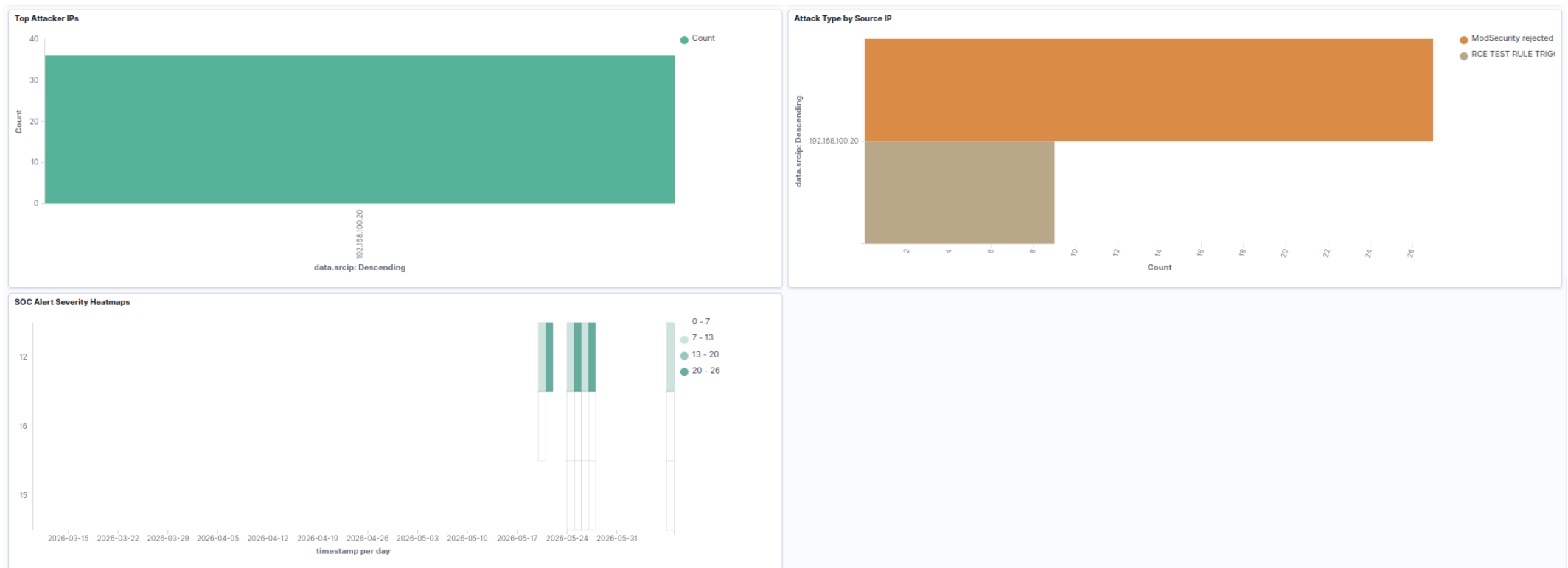


Figure 12.3 – Threat Analysis Dashboard

Figure 12.3 presents the Threat Analysis Dashboard developed within SOCSshield. The dashboard correlates attacker source IP addresses with detected attack types and severity levels. By visualizing attack attribution and threat distribution, analysts can quickly identify malicious sources, investigate attack behavior, and prioritize incident response activities.

13. Results

13.1 Overview

The SOCSshield platform was successfully implemented and tested in a controlled laboratory environment. Multiple web application attack scenarios were executed against the DVWA application to validate the detection, monitoring, threat intelligence, alerting, and automated response capabilities of the platform.

The project successfully demonstrated the integration of ModSecurity Web Application Firewall, Wazuh SIEM, VirusTotal Threat Intelligence, Email Alerting, MITRE ATT&CK Mapping, and Active Response mechanisms within a unified Security Operations Center (SOC) monitoring environment.

Testing confirmed that SOCSshield was capable of detecting malicious activities, enriching alerts with threat intelligence, notifying analysts through email, and automatically responding to selected high-severity attacks.

13.2 Attack Detection Results

SOCSshield successfully detected multiple web application attack types generated during testing. Custom Wazuh rules and ModSecurity signatures were used to identify malicious requests and generate security alerts.

The platform successfully detected:

- “Cross-Site Scripting (XSS)”
- “SQL Injection (SQLi)”
- “Remote Code Execution (RCE)”
- “Command Injection Attempts”

Attack events were recorded within the Wazuh dashboard and visualized through custom monitoring dashboards.

- **Attack Detection Results**

Attack Type	Detection Status
1. XSS	Successfully Detected
2. SQL Injection	Successfully Detected
3. RCE	Successfully Detected
4. Command Injection	Successfully Detected

13.3 MITRE ATT&CK Mapping Results

SOCShield successfully mapped detected attacks to relevant MITRE ATT&CK tactics and techniques. This provided additional context to analysts and improved understanding of attacker behaviour.

The mapping process enabled classification of attack activities according to industry-recognized threat intelligence standards.

- **Table – MITRE ATT&CK Mapping Results**

Attack	MITRE Technique
1. SQL Injection	Exploit Public-Facing Application (T1190)
2. RCE	Command and Scripting Interpreter (T1059)
3. Web Attacks	Initial Access Techniques

13.4 Active Response Results

Active Response testing confirmed that Wazuh successfully executed the firewall-drop mechanism upon detection of high-severity attack events. Malicious source IP addresses were automatically blocked, reducing the potential impact of repeated attack attempts.

Response actions were logged within the Wazuh platform and visualized through the Active Response Dashboard.

• **Table – Active Response Results**

Feature	Status
1. Firewall-Drop Execution	Successful
2. IP Blocking	Successful
3. Response Logging	Successful
4. Dashboard Visualization	Successful

13.5 Threat Intelligence Results

VirusTotal Threat Intelligence integration was successfully configured and validated. Security indicators were submitted for reputation analysis and enrichment within Wazuh.

During testing, VirusTotal enrichment alerts were generated successfully. Public API rate limitations were observed during high-frequency testing, demonstrating expected operational constraints associated with the public VirusTotal API tier.

• **Table – Threat Intelligence Results**

Capability	Status
1. VirusTotal Integration	Successful
2. Threat Enrichment	Successful
3. Reputation Lookup	Successful
4. API Communication	Successful

13.6 Email Alerting Results

Email alerting functionality was successfully implemented using Postfix and Gmail SMTP services. Security alerts generated by Wazuh were automatically delivered to the analyst mailbox.

Testing confirmed end-to-end notification delivery for detected security events, improving analyst awareness and reducing incident response delays.

• **Table – Email Alerting Results**

Feature	Status
1. Email Notification	Successful
2. SMTP Relay	Successful
3. Alert Delivery	Successful
4. Analyst Notification	Successful

13.7 Dashboard and Monitoring Results

Multiple custom dashboards were developed to provide centralized visibility into attack activity, attacker behaviour, security event severity, and automated response actions.

The dashboards successfully visualized:

- “Attack timelines”
- “Attack type distribution”
- “Severity analysis”
- “Attacker source IP analysis”
- “Firewall block activity”
- “Active Response execution”

13.8 Overall Project Outcome

The implementation of SOCSshield successfully achieved all project objectives. The platform demonstrated the ability to detect web application attacks, enrich alerts with threat intelligence, automate incident response actions, notify analysts through email, and provide comprehensive monitoring through custom dashboards.

The project validates the effectiveness of combining Web Application Firewalls, SIEM technologies, threat intelligence feeds, and automated response mechanisms to create a practical Security Operations Center solution for modern cybersecurity environments.

14. Challenges Encountered

14.1 Overview

The development of SOCSshield involved the integration of multiple cybersecurity technologies including Wazuh SIEM, ModSecurity WAF, NGINX, DVWA, VirusTotal, Postfix, and custom dashboards. Due to the complexity of integrating multiple security components within a single environment, several technical and operational challenges were encountered throughout the project lifecycle.

These challenges ranged from system deployment and networking issues to log parsing, alert correlation, threat intelligence integration, automated response execution, and dashboard development. Resolving these issues required extensive troubleshooting, testing, and configuration refinement.

14.2 Virtual Environment and Network Configuration Challenges

Challenge

- VM-to-VM connectivity problems
- Incorrect network adapter configurations
- IP addressing conflicts
- NAT and Host-Only network troubleshooting
- Service accessibility issues between systems

Resolution

Network interfaces were reconfigured and connectivity was validated using tools such as ping, netstat, ss, and tcpdump. Static IP addressing was implemented to ensure stable communication between project components.

Outcome

Reliable communication was established across all systems, enabling attack simulation and centralized monitoring.

14.3 Wazuh Installation and Service Configuration Challenges

Challenge

During the deployment of Wazuh Manager and associated services, several service initialization and configuration issues were encountered. Some components failed to start correctly due to dependency and configuration inconsistencies.

Resolution

Service logs were analyzed using journalctl and Wazuh log files. Configuration errors were corrected and services were restarted and validated after each modification.

Outcome

Wazuh Manager and supporting services operated successfully and provided centralized security monitoring.

14.4 ModSecurity and Log Collection Challenges

Challenge

Although ModSecurity successfully detected malicious requests, ensuring that logs were correctly collected and processed by Wazuh required additional configuration.

Issues included:

- Incorrect log file paths
- Missing log collection configuration
- Event parsing inconsistencies
- Incomplete attack details in generated alerts

Resolution

Log collection settings were refined and custom decoders were implemented to improve event parsing and extraction of attack-related information.

Outcome

ModSecurity events were successfully forwarded to Wazuh for analysis and correlation.

14.5 Custom Rule Development Challenges

Challenge

Default Wazuh rules did not provide the desired level of attack categorization required for SOCSshield. Specific detection requirements for XSS, SQL Injection, RCE, and Command Injection attacks required the development of custom rules.

Resolution

Custom rule IDs, severity levels, and alert descriptions were created and tested using multiple attack scenarios.

Outcome

The platform successfully generated attack-specific alerts with improved visibility and classification.

14.6 Active Response Source IP Extraction Challenges

Challenge

During Active Response implementation, the firewall-drop mechanism initially failed because the source IP address could not be extracted from generated alerts.

Observed error:

```
active-response/bin/firewall-drop:  
Cannot read 'srcip' from data
```

Resolution

Alert structures, decoders, and rule mappings were modified to ensure correct extraction of attacker IP addresses.

Outcome

Active Response successfully executed firewall-drop actions and blocked malicious source IP addresses.

14.7 VirusTotal Threat Intelligence Integration Challenges

Challenge

Integrating VirusTotal with Wazuh required API configuration and validation. During testing, public API limitations resulted in rate-limit errors when multiple requests were generated within a short period.

Observed alert:

```
VirusTotal:  
Error: Public API request rate limit reached
```

Resolution

Request frequency was adjusted and testing procedures were modified to remain within VirusTotal public API limitations.

Outcome

Threat intelligence enrichment functionality was successfully validated.

14.8 Email Alerting Challenges

Challenge

Initial attempts to send security notifications failed due to SMTP configuration issues and Gmail authentication restrictions.

Issues included:

- Incorrect SMTP settings
- Authentication failures
- Mail delivery errors
- Gmail security restrictions

Resolution

Postfix was configured as the SMTP relay service and Gmail App Password authentication was implemented.

Outcome: Security alerts were successfully delivered to the analyst mailbox.

14.9 Dashboard Development Challenges

Challenge

Building meaningful SOC dashboards required extensive experimentation with Wazuh visualization tools and field mappings.

Challenges included:

- Selecting relevant data fields
- Creating attack correlation views

- Designing severity heatmaps
- Visualizing attacker IP activity
- Developing response monitoring dashboards

Resolution

Multiple dashboard iterations were created and refined until useful operational visualizations were achieved.

Outcome

Custom SOC dashboards successfully visualized attack activity, threat trends, and response actions.

14.10 Integration and Correlation Challenges

Challenge

SOCShield combines several independent technologies that were not originally designed to work together as a single platform.

Integrated components included:

- Kali Linux
- Burp Suite
- DVWA

- NGINX
- ModSecurity
- Wazuh
- VirusTotal
- Postfix

Ensuring consistent data flow between these components required extensive troubleshooting.

Resolution

Log collection paths, rule mappings, integrations, and communication settings were repeatedly validated and adjusted.

Outcome

Successful end-to-end event correlation was achieved.

14.11 Testing and Validation Challenges

Challenge

Validating every security capability required generating realistic attack scenarios without disrupting the environment.

Challenges included:

- Reproducing attack events consistently
- Triggering specific detection rules
- Validating Active Response execution
- Testing threat intelligence enrichment

- Confirming email delivery

Resolution

Controlled attack simulations using DVWA and Burp Suite were repeatedly executed and monitored through Wazuh.

Outcome

All major platform features were successfully tested and validated.

15. Future Enhancements

15.1 Overview

Although SOCSshield successfully demonstrates web attack detection, threat intelligence enrichment, automated response, and security monitoring capabilities, several enhancements can be implemented in future versions to improve scalability, intelligence, automation, and enterprise readiness.

The following enhancements represent potential improvements that would further strengthen the platform and expand its capabilities within modern Security Operations Center (SOC) environments.

15.2 Multi-Agent Deployment

Proposed Enhancement

The current implementation primarily focuses on monitoring a single web application environment. Future versions of SOCSshield can support multiple Wazuh agents deployed across various servers, endpoints, and network devices.

Benefits

- Centralized monitoring of multiple assets
- Improved enterprise visibility
- Enhanced threat detection coverage
- Scalable architecture

15.3 Integration with Additional Threat Intelligence Sources

Proposed Enhancement

Currently, VirusTotal is used for threat intelligence enrichment. Future implementations can integrate multiple threat intelligence feeds to provide broader context and improved indicator reputation analysis.

Potential Integrations

- AbuseIPDB
- AlienVault OTX
- MISP

- URLHaus
- OpenPhish

Benefits

- Improved threat intelligence coverage
- Better indicator correlation
- Enhanced threat hunting capabilities

15.4 SOAR Integration

Proposed Enhancement

Future versions of SOCSHield can integrate Security Orchestration, Automation, and Response (SOAR) platforms to automate incident investigation and response workflows.

Possible Platforms

- Shuffle SOAR
- TheHive
- Cortex
- Splunk SOAR

Benefits

- Automated playbooks
- Faster incident response
- Reduced analyst workload
- Improved operational efficiency

15.5 Advanced Active Response Mechanisms

Proposed Enhancement

The current implementation uses firewall-drop actions to block malicious source IP addresses. Future versions can implement more advanced response actions.

Possible Actions

- Dynamic firewall updates
- User account isolation
- Automated quarantine
- Endpoint containment
- Web server rule updates

Benefits

- Stronger threat containment
- Reduced attacker dwell time Improved response effectiveness

16. Conclusion

The increasing frequency and sophistication of web-based cyber-attacks have made continuous monitoring, threat detection, and rapid incident response essential requirements for modern organizations. Traditional security monitoring approaches often struggle to provide the visibility, intelligence, and automation necessary to effectively identify and respond to emerging threats.

SOCShield was developed as an intelligent Security Operations Center (SOC) platform designed to address these challenges through the integration of open-source security technologies. The project successfully combined Wazuh SIEM, ModSecurity Web Application Firewall (WAF), NGINX, DVWA, VirusTotal Threat Intelligence, MITRE ATT&CK Mapping, Active Response mechanisms, Email Alerting, and custom security dashboards into a unified monitoring and response environment.

Throughout the project, multiple web application attack scenarios including Cross-Site Scripting (XSS), SQL Injection (SQLi), Remote Code Execution (RCE), and Command Injection were successfully simulated and detected. Security events generated by these attacks were collected, analyzed, correlated, and visualized

through centralized dashboards, providing valuable insights into attacker behavior and security posture.

The implementation of VirusTotal integration enhanced threat visibility through reputation-based threat intelligence enrichment, while MITRE ATT&CK Mapping provided standardized classification of attack techniques and tactics. In addition, automated Active Response capabilities demonstrated the ability to contain malicious activity by dynamically blocking attacker IP addresses through firewall-drop actions. Real-time email notifications further improved analyst awareness by ensuring immediate delivery of critical security alerts.

The custom dashboards developed within SOCSshield successfully transformed raw security logs into actionable intelligence by providing visibility into attack trends, severity distributions, attacker source analysis, and automated response activities. These visualizations improved monitoring efficiency and supported effective threat hunting and incident investigation.

The successful completion of SOCSshield demonstrates that a practical and effective Security Operations Center can be built using open-source technologies while maintaining strong detection, monitoring, threat intelligence, and response capabilities. The project provided valuable hands-on experience in SIEM deployment, web application security monitoring, threat intelligence integration, automated response implementation, dashboard development, and SOC operations.

In conclusion, SOCSshield successfully achieved all defined project objectives and demonstrated a comprehensive approach to modern web application threat detection, analysis, and response. The platform serves as a practical example of how integrated security technologies can be leveraged to enhance organizational cybersecurity capabilities and strengthen overall security operations.

